



Software Testing Types

Purpose of this document

The main purpose of this document is to perform different types of Software Testing on the Code Informatics Website along with screenshots.

Contents

Unit Testing:	3
Cross Browser Testing:	3
W3C Validation:.....	4
Black Box Testing:	5
White Box Testing:	6
Integration Testing:	7
Functional Testing:	9
Performance Testing:.....	10
System Testing:	10
End to End Testing:	11
Sanity Testing:	11
Regression Testing:	12
Load Testing:	12
Stress Testing:	13
Usability Testing:	13
User Acceptance Testing (UAT):	14
Conclusion:	14

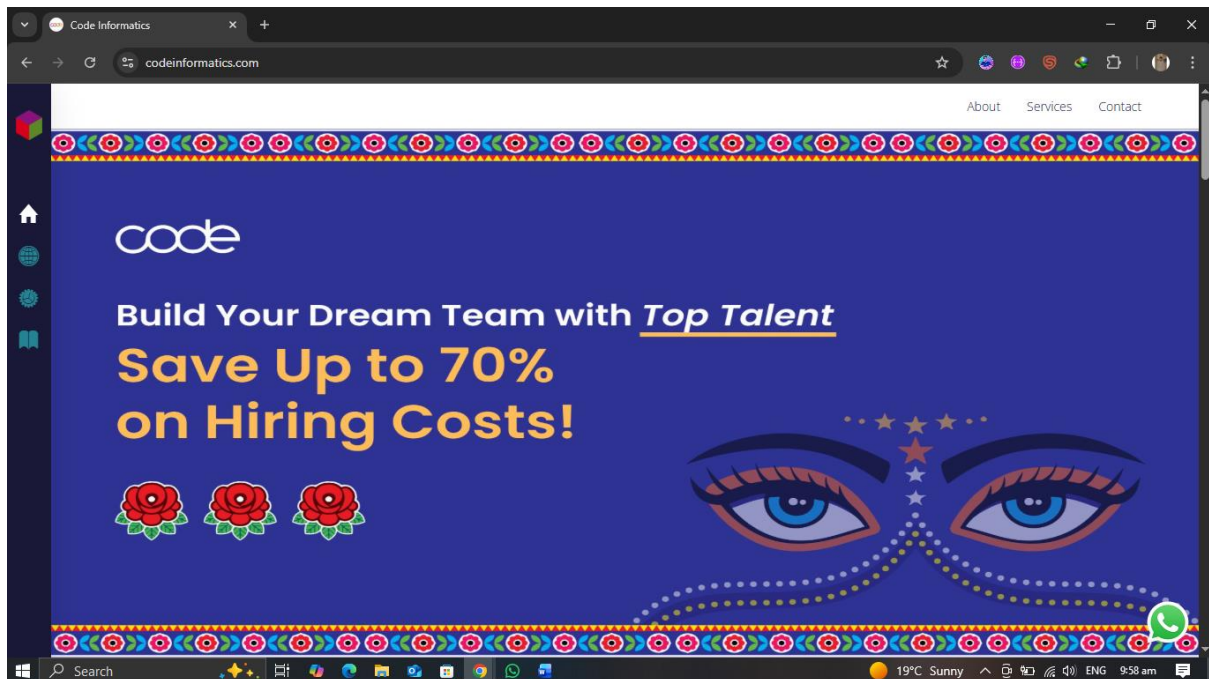
Unit Testing:

Unit Testing is done by the **developer**. The developer tests his **code** before going into further testing to make it **more efficient**.

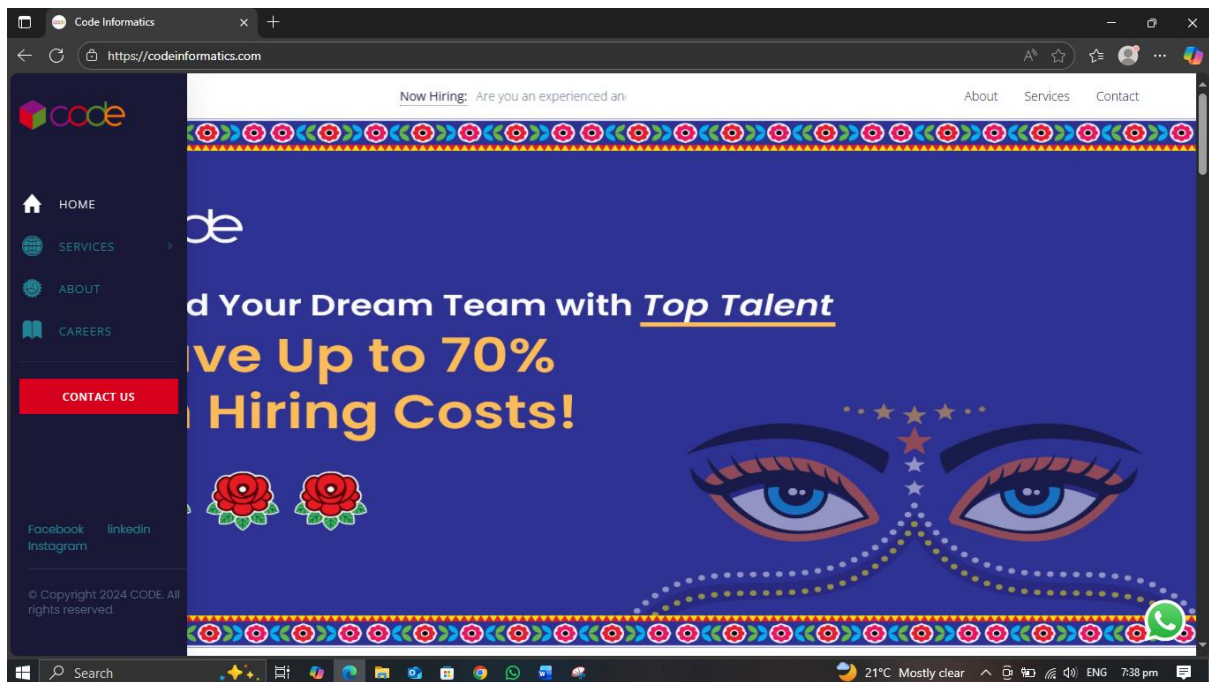
Cross Browser Testing:

To make sure the website looks and behaves **the same on all browsers**, we do cross browser testing on different browsers.

Google Chrome:



Microsoft Edge:



W3C Validation:

W3C validation is a **semi automation** testing tool, in this testing we will check the flow of our **HTML** and **CSS** code. We go to <https://validator.w3.org/> this site and paste the **URL** of our site and then we get **3 types** of results.

The validator will show:

- ☒ Passed (No Errors)
- ☐ Warnings (not critical but worth fixing)
- ☐ Errors (must fix issues)

1. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 6, column 3; to line 6, column 26
`<head><meta charset="UTF-8" /><me`
2. **Warning** Consider avoiding viewport values that prevent users from resizing documents.
From line 7, column 3; to line 7, column 128
`F-8" /><meta content="width=device-width, initial-scale=1.0, minimum-scale=1.0, maximum-scale=1.0, user-scalable=no" name="viewport"><1`
3. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 9, column 3; to line 9, column 54
`port"><link rel="profile" href="http://gmpg.org/xfn/11" /><li`
4. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 10, column 3; to line 10, column 71
`/11" /><link rel="pingback" href="https://codeinformatcs.com/xmlrpc.php" /><t`
5. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 13, column 1; to line 13, column 50
`s</title><meta name="robots" content='noindex, nofollow' /><sty`
6. **Error** CSS: `contain-intrinsic-size`: Property `contain-intrinsic-size` doesn't exist.
At line 14, column 93
`: 3000px 1500px ;</style><li`

7. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 15, column 2; to line 15, column 58
`</style><link rel="dns-prefetch" href="//fonts.googleapis.com" /><link`
8. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 16, column 1; to line 16, column 130
`s.com" /><link rel="alternate" type="application/rss+xml" title="Code Informatics » Feed" href="https://codeinformatcs.com/feed/" /><link`
9. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 17, column 1; to line 17, column 148
`feed/" /><link rel="alternate" type="application/rss+xml" title="Code Informatics » Comments Feed" href="https://codeinformatcs.com/comments/feed/" /><scri`
10. **Warning** The `type` attribute is unnecessary for JavaScript resources.
From line 18, column 1; to line 18, column 31
`feed/" /><script type="text/javascript"> /* <`
11. **Info** Trailing slash on void elements [has no effect](#) and [interacts badly with unquoted attribute values](#).
From line 25, column 1; to line 25, column 183
`</script><link rel="stylesheet" id="jquery.prettyphoto-css" href="https://codeinformatcs.com/wp-content/plugins/wp-video-lightbox/css/prettyPhoto.css?ver=6.8.3" type="text/css" media="all" /><link`

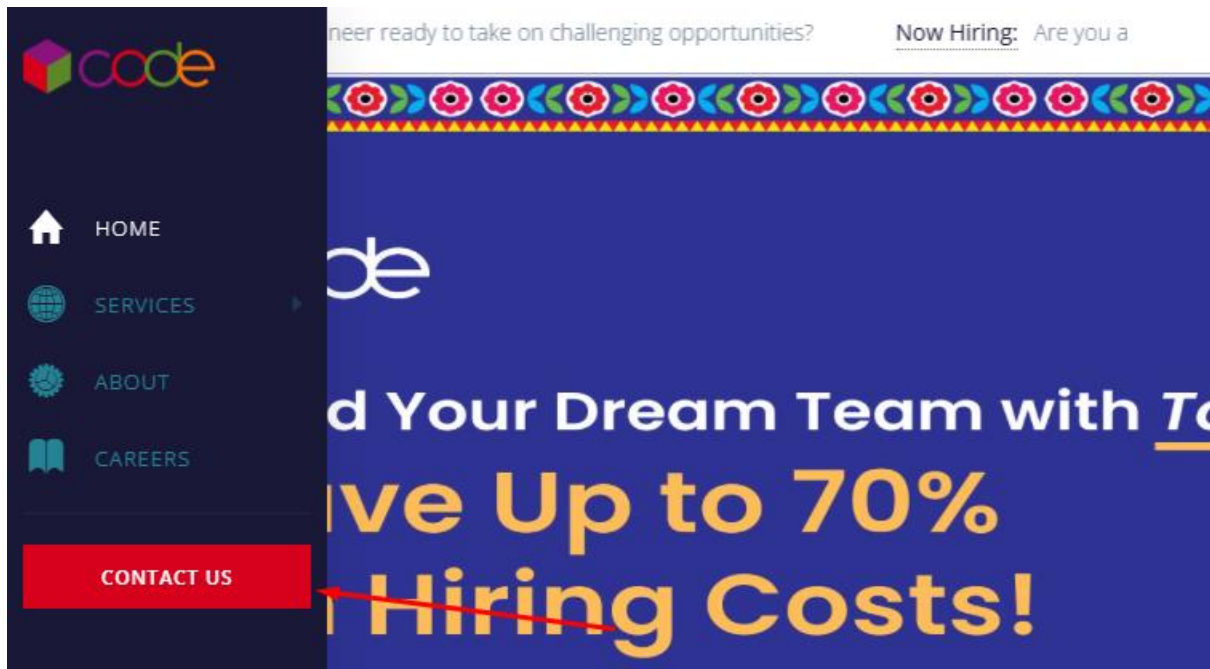
Even small **W3C errors** can affect how your website looks on different browsers. Fixing them makes the website **more stable, accessible, and SEO-friendly**.

Black Box Testing:

In block box testing we test the **functionality** from the user's point of view, you **don't look at the code**, you just test **inputs and outputs** to see if the website behaves correctly.

We focus only on what the user can see and do not on how it's built.

Always test both **positive (valid data)** and **negative (invalid data)** scenarios.

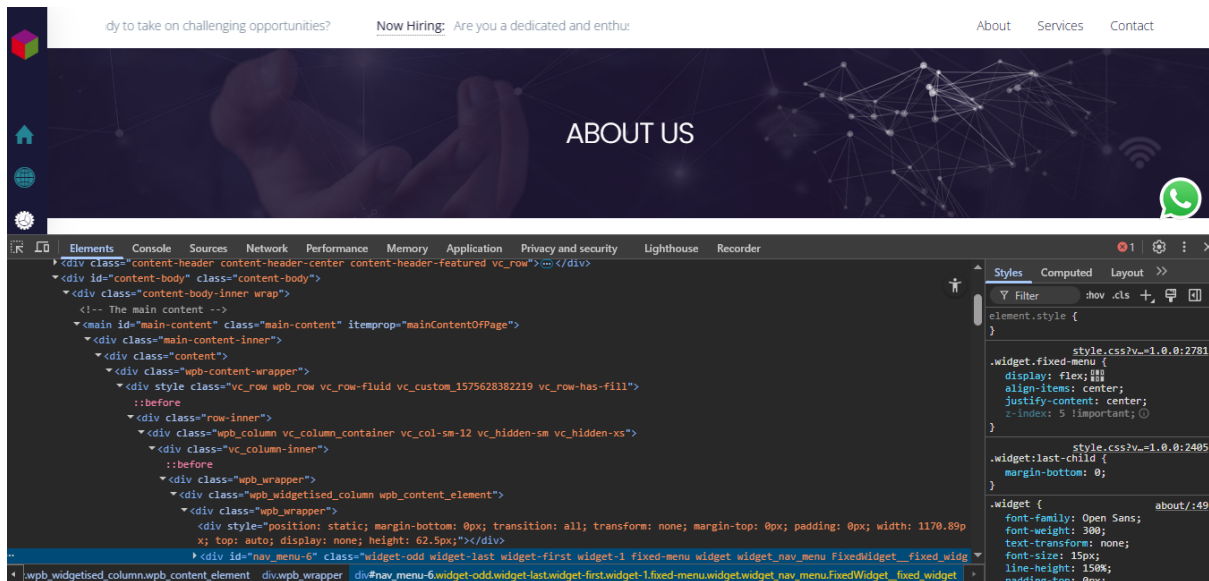


After clicking on **contact us**, we are redirect to contact us page, This **functionality** is **working** according to our requirement.

 The image shows a contact form on a website. The form is white and centered on a light blue background. It has a dark blue sidebar on the left with a logo and navigation icons. The form contains several input fields and dropdown menus. The fields are: 'Full name *' with a placeholder 'e.g., John Doe', 'Organization *' with a placeholder 'Company Name', 'Work email address *' with a placeholder 'name@company.com', 'Phone number' with a placeholder 'Full Number (incl. prefix)', 'Company size *' with a dropdown menu showing 'Please Select', and 'What is your inquiry about? *' with a dropdown menu showing 'Please Select'. At the bottom is a large text area labeled 'How we can help you? *' with a placeholder 'Let us know what you need.'. A green WhatsApp icon is visible in the bottom right corner of the page.

White Box Testing:

In white box testing we test the internal **logic** from the **developer's** point of view. Make sure every part of the **code logic works correctly** and no lines are left untested.

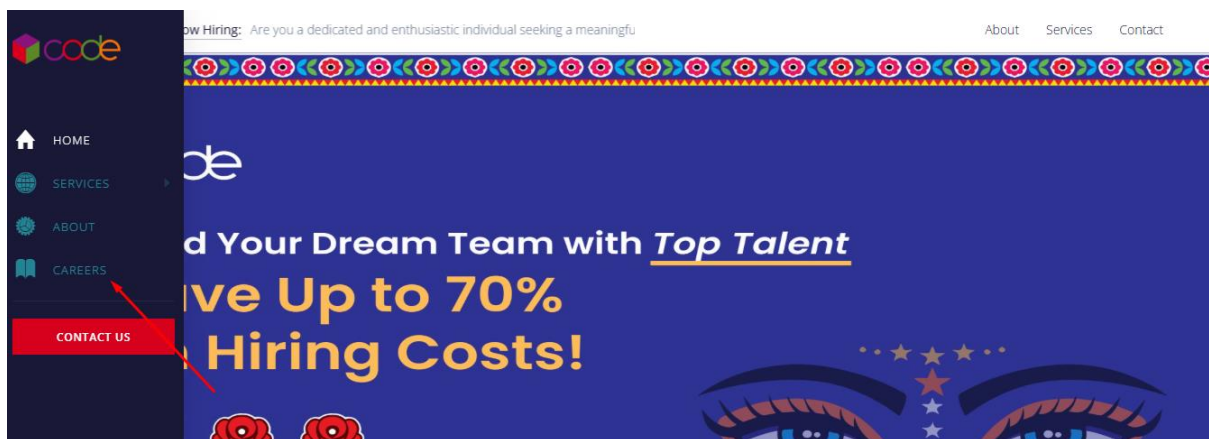


Here we are checking the **code** behind the **About Us** page of the Code Informatics Website.

Integration Testing:

In integration testing we check how different **modules** or **parts** of the system work **together**. In simple we check **connected parts of the system** work properly together.

First, we have a **Careers module** on home page. After clicking on careers, we will be **redirected** to careers page then we open and explore in details.



Now we can see there is multiple jobs available on **careers page**, we click on **more details** in **Frontend Developer** section.

Frontend Developer (React/Next.js)	Technical Full Time hybrid, remote	MORE DETAILS →
Marketing Intern		MORE DETAILS →
HR Intern		MORE DETAILS →
Software Quality Assurance (SQA) Engineer	Technical Full Time remote	MORE DETAILS →

Then, there is another **module/functionality** is added after clicking on **more details** that is detail description related to job, which means the combined **functionality** of both modules are working fine.

Frontend Developer (React/Next.js)

 by Code HR • September 25, 2025

Location: Remote / Hybrid
Employment Type: Full-Time
Experience Level: Mid to Senior

About the Role

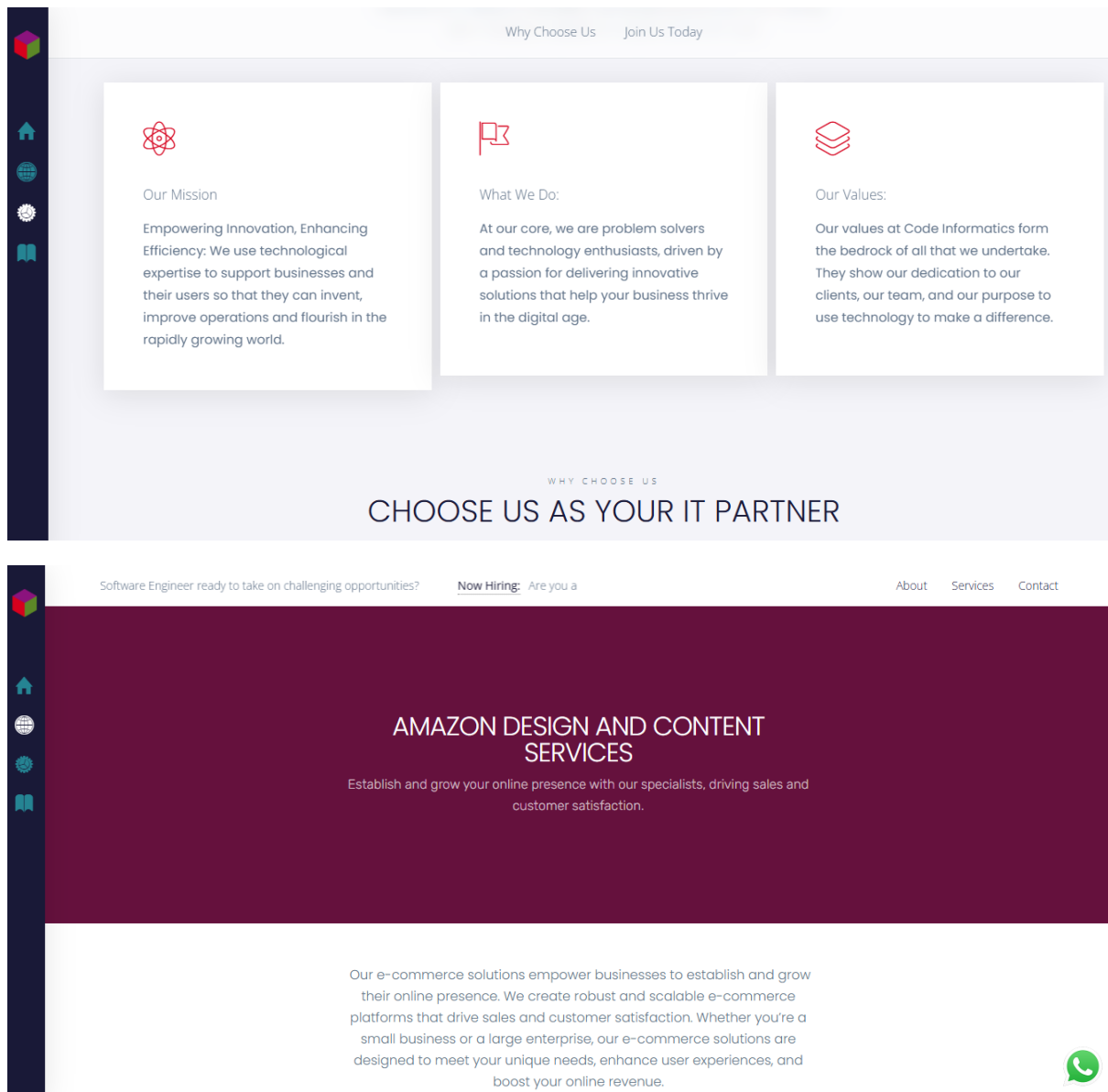
We are looking for a skilled **Frontend Developer** with solid experience in **Next.js** and **React** to join our team in building a modern, scalable web application powered by AWS. You'll be responsible for delivering performant and accessible UI, working closely with backend and DevOps teams to ensure seamless integration and deployment.

Key Responsibilities

- Develop, test, and maintain user-facing features using **React and Next.js**
- Implement **Server-Side Rendering (SSR)** and **Static Site Generation (SSG)** strategies for optimal performance and SEO
- Work with backend APIs and integrate with services hosted on **AWS**
- Optimize application performance, accessibility, and responsiveness across devices and browsers
- Collaborate with UX/UI designers to bring mockups to life with pixel-perfect accuracy
- Participate in code reviews, CI/CD pipeline workflows, and agile ceremonies (stand-ups, sprint planning, etc.)
- Ensure secure handling of user data and interactions in the frontend
- Integrate with authentication systems (**AWS Cognito**) and manage user sessions effectively

Functional Testing:

In functional testing we test the **functions** and **features** of the systems to make sure they work correctly against **SRS**.



Performance Testing:

In performance testing we test the **speed, stability, and responsiveness** of the system.

Now Hiring: Are you an experienced and motivated Senior Software E

About Services Contact

CONTACT US

We'd love to discuss how we can help you streamline your business with the Code Informatics Managed Service Suite. Contact us for a no-obligation conversation. A member of our team will get in touch with you within a few business hours.

Full name *

e.g., John Doe

Organization *

Company Name

Work email address *

name@company.com

Phone number

Full Number (intl. prefix)

WhatsApp icon

System Testing:

In system testing we test the **entire** system to ensure it **meets** all **requirements** and **works** as expected.



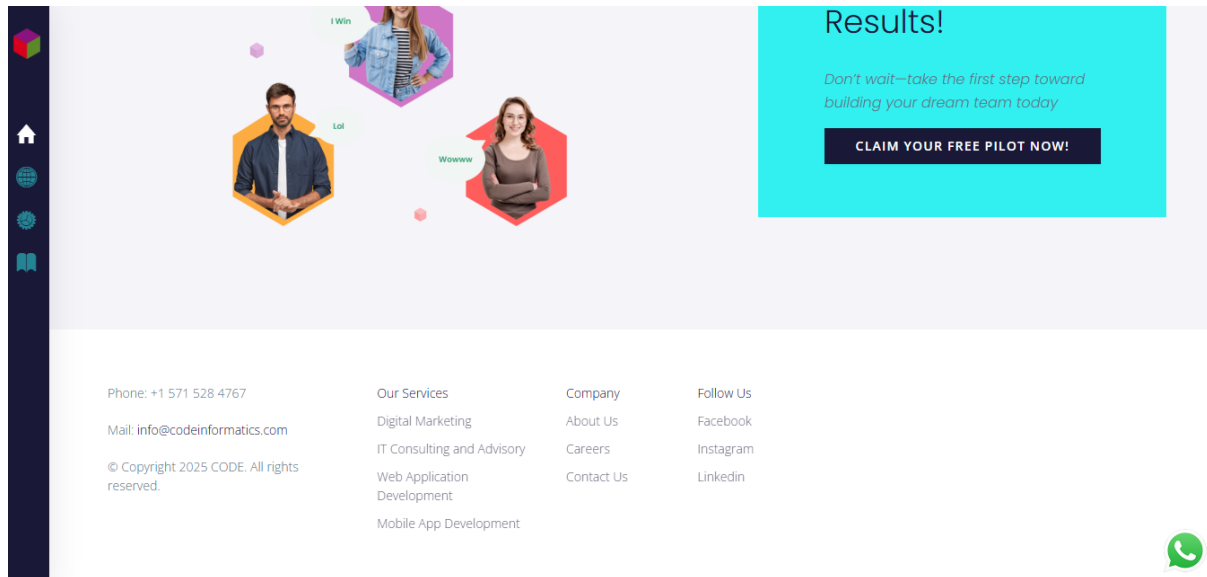
The entire module we tested as:

1. Home
2. Services
3. About
4. Careers
5. Contact us
6. Social media links

7. WhatsApp

End to End Testing:

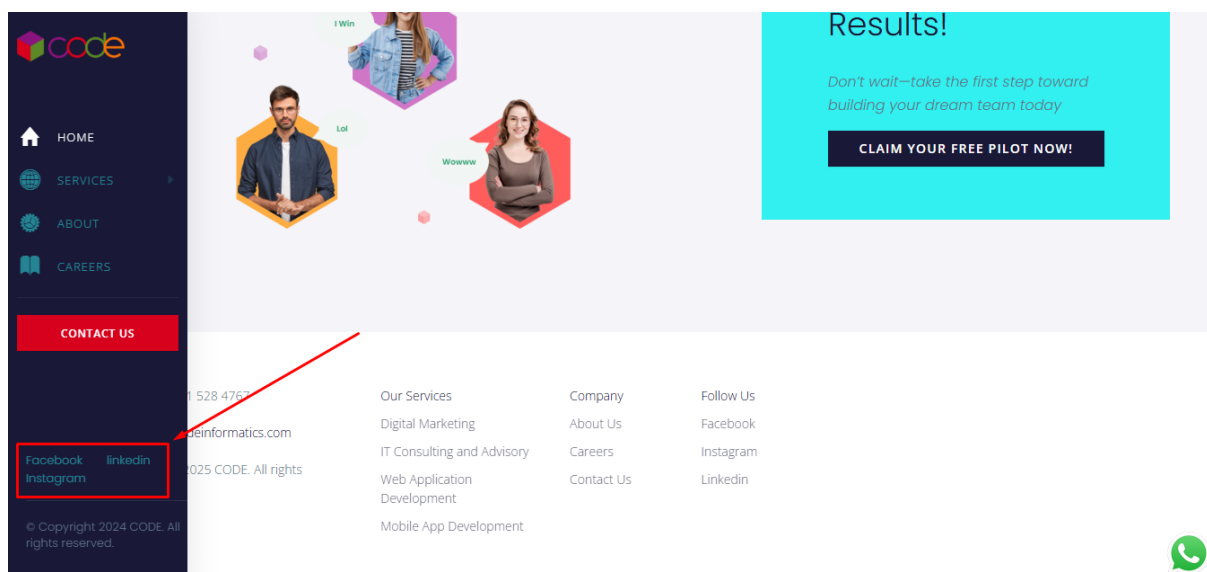
In end-to-end testing we test the **entire process** from the **beginning** to the **end** to ensure everything works **smoothly**.



Sanity Testing:

In sanity testing a **quick** check to make sure **everything** is **working** fine after small changes.

Let's suppose we have added **social media links** in our system and release a **new build** of the Website, we will perform the **Sanity Testing** whether the System is running **fine** or **not**, otherwise we will not be able to test the system further.



Regression Testing:

In regression testing we check that **old** features still **work** after new **update** or **fixes**.

Let's Suppose, we have added a new Module related to WhatsApp in the Code Informatics Website, now we have to test the other modules from the start as well.



Load Testing:

In load testing we test how the system **performs** under **normal** or **expected** load.

Stress Testing:

In stress testing we test how system's **breaking point** by putting it under **extreme pressure**.

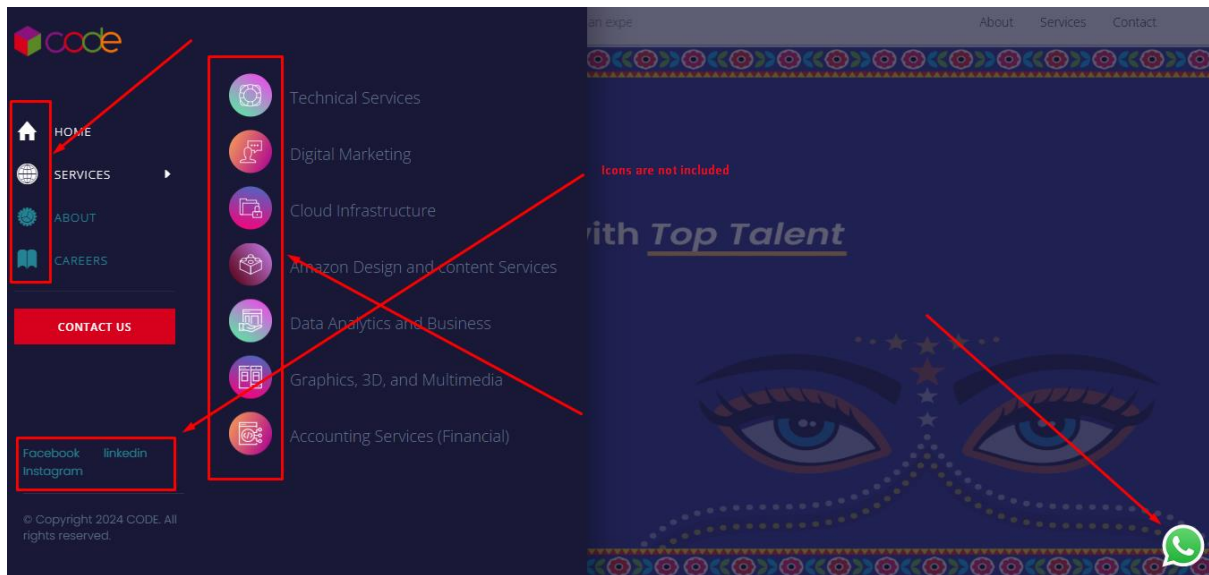
In this Website, we will test the **Contact Us** page by giving the unfavourable inputs.

Full name *	Organization *
vcv	cvc
Work email address *	Phone number
sds232	cvc
The e-mail address entered is invalid.	The telephone number is invalid.
Company size *	What is your inquiry about? *
2-19	Please Select
How we can help you? *	
Let us know what you need.	

Usability Testing:

In usability testing we test how **simple**, **clear**, and **comfortable** the app is for users to use.

In this particular Website, Icons are added to make it easier for user to **understand** the purpose of each element.



User Acceptance Testing (UAT):

In UAT, an **end user** tests the system to confirm it's **ready for release**.

Conclusion:

Through the testing process of the **Code Informatics** website, various testing types were performed, including **Unit Testing, Integration Testing, System Testing, Functional Testing, Performance Testing, Load Testing, Stress Testing, Sanity Testing, Regression Testing, and End-to-End Testing**.

Each testing type played a specific role in ensuring that the website is **stable, functional, efficient, and user-friendly**.

- **Unit and Integration Testing** helped confirm that the internal modules and connections work correctly.
- **Functional and System Testing** verified that all features perform as expected.
- **Performance, Load, and Stress Testing** ensured that the system remains reliable under normal and heavy usage.
- **Sanity and Regression Testing** helped maintain quality after updates or bug fixes.
- Finally, **End-to-End Testing** validated the complete user journey from start to finish.

Overall, these tests confirmed that the **Code Informatics website** performs smoothly across different browsers and devices, delivering a **consistent and high-quality user experience**.